RELATÓRIO DE IMPLEMENTAÇÃO / VERSÃO 0.2

MicroLearn

Sprint 2 — Cursos, aulas e matrículas

Objetivo do incremento

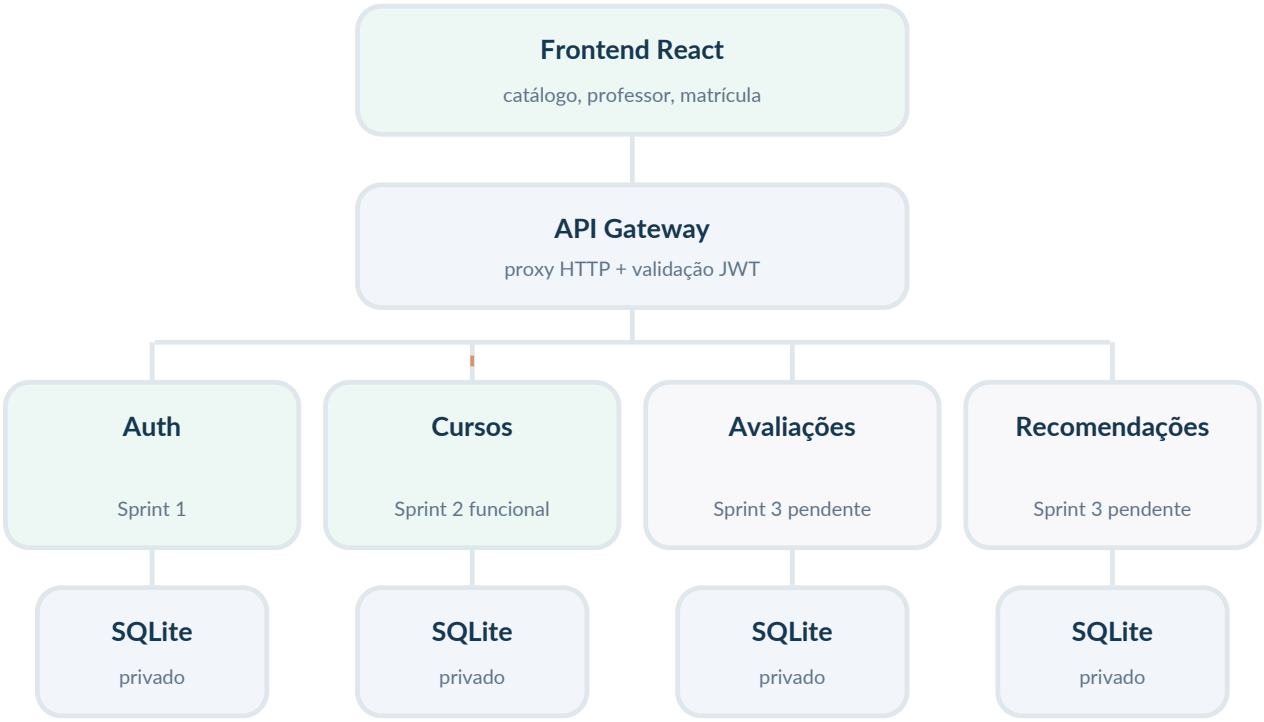
Implementar o primeiro fluxo acadêmico completo do Ambiente Virtual de Aprendizagem: cadastro e publicação de cursos por professores, criação de aulas, consulta do catálogo, matrícula de alunos e acesso restrito ao conteúdo. O trabalho preserva a Gateway e a autenticação da Sprint 1.

Entrega	Estado observado
Cursos e aulas (CRUD)	Implementado e validado com testes automatizados e HTTP real
Publicação e catálogo	Somente cursos publicados aparecem na consulta pública
Matrículas	Cadastro único por aluno/curso, acesso a aulas condicionado ao vínculo
Perfis e autorização	Autor de curso e aluno matriculado; identidade revalidada no Auth
Frontend React	Novas telas e componentes; sintaxe verificada, build pendente
Docker Compose	Configuração atualizada; execução em Docker não validada aqui

Status de validação: 16/16 testes Python passaram; smoke HTTP entre cinco processos passou; 19 arquivos JS/JSX sem erros sintáticos. O build do React depende da instalação de pacotes npm, indisponível no ambiente de geração.

1. Arquitetura da Sprint 2

A aplicação possui cinco processos HTTP independentes: API Gateway, Autenticação, Cursos, Avaliações e Recomendações. O frontend React é um sexto componente de implantação no Docker Compose. Cada domínio de negócio mantém seu próprio SQLite.



A consulta à conta ativa e ao papel do usuário é realizada pelo Courses Service via HTTP GET /auth/me, usando o token recebido. Assim, Cursos não compartilha nem consulta diretamente o banco da Autenticação.

1.1. Persistência de Cursos

Tabela	Campos relevantes / restrições
courses	UUID, título, descrição, categoria, teacher_id, status draft/published, timestamps
lessons	UUID, course_id (FK), título, texto, material_url, posição; exclusão em cascata
enrollments	UUID, course_id (FK), student_id, data; UNIQUE(course_id, student_id)

2. Contratos REST e regras de acesso

Todos os caminhos abaixo usam o prefixo /api/v1 na API Gateway. A Gateway valida JWT em rotas privadas; o Courses Service reforça as regras dentro de seus próprios endpoints.

Método e caminho	Permissão / comportamento
GET /courses	Público; lista somente cursos publicados
POST /courses	Professor; cria curso como draft
GET /courses/owned	Professor; lista apenas seus cursos
GET /courses/enrollments/mine	Aluno; lista suas matrículas
GET /courses/{id}	Autenticado; rascunho visível apenas ao autor
PATCH, DELETE /courses/{id}	Professor autor; editar/publicar/excluir
POST /courses/{id}/enrollments	Aluno; matricular-se em curso publicado
GET /courses/{id}/lessons	Professor autor ou aluno matriculado
POST /courses/{id}/lessons	Professor autor; criar aula
PATCH, DELETE /courses/{id}/lessons/{lesson_id}	Professor autor; alterar/remover aula

2.1. Regras verificadas

- Uma matrícula duplicada retorna HTTP 409 e permanece única no banco por restrição UNIQUE.
- Um aluno não pode criar cursos ou aulas; um professor não pode alterar curso de outro.
- Rascunhos não aparecem no catálogo nem aceitam matrículas.
- O aluno precisa estar matriculado para acessar as aulas do curso publicado.
- A remoção de um curso também remove aulas e matrículas relacionadas no banco do domínio.
- A alteração do perfil no Auth Service interrompe privilégios antigos, mesmo com JWT ainda válido.

3. Interface baseada em componentes

Novos componentes React foram adicionados sem duplicar a lógica de acesso à API. Todos enviam requisições ao mesmo cliente HTTP centralizado. A integração visual definitiva deverá ser confirmada no navegador.

Componente	Papel no frontend
CourseCard	Cartão reutilizado no catálogo, nas matrículas e na área do professor
CourseForm	Formulário reutilizado para criação e alteração de dados do curso
LessonEditor	Formulário de aula reutilizado em inclusão e edição
apiClient / coursesApi	Agrupa operações HTTP da Gateway e tratamentos de erro
AuthContext / ProtectedRoute	Guarda a sessão didática e restringe a navegação privada

3.1. Telas adicionadas

- /cursos: consulta pública de cursos publicados;
- /professor/cursos: criação de cursos e listagem de autoria;
- /cursos/:id: edição/publicação de curso, aulas e matrícula conforme o perfil;
- /meus-cursos: cursos nos quais o aluno está matriculado.

3.2. Fluxo funcional demonstrado

Administrador promove um aluno ao papel de professor → professor cria curso → adiciona aula → publica → aluno autenticado consulta o catálogo → realiza matrícula → acessa aulas. O fluxo também foi automatizado por requisições HTTP reais no smoke test.

3.3. Fora do escopo atual

Avaliações e Recomendações continuam com endpoints de negócio HTTP 501. O gerenciamento de arquivos físicos, uploads de vídeo e videoconferências não faz parte do MVP. Links de materiais externos e conteúdo textual atendem ao escopo previsto para aulas.

4. Testes e evidências de execução

Os resultados correspondem ao código Python entregue. Testes de subprocessos usam bancos temporários e requisições HTTP de ponta a ponta entre Gateway e microsserviços.

Teste ou verificação	Resultado
pytest -q	16 testes aprovados, cobrindo Auth, Gateway, domínios e Cursos
smoke_processes.py	Cinco processos locais com saúde OK
Criação, publicação e catálogo	HTTP 201/200; apenas publicados disponíveis
Cadastro de aula e matrícula	HTTP 201; aluno matriculado acessa conteúdo
Matrícula duplicada	HTTP 409
Aluno tentando criar curso	HTTP 403
Revogação de papel de professor	Operação bloqueada com HTTP 403 após mudança de papel
Parser TypeScript de JSX/JS	19 arquivos analisados; sem erros de sintaxe
YAML Compose	6 serviços reconhecidos pela leitura de configuração
Docker / build React	Não executados; verificações ainda pendentes no Windows

4.1. Evidências preservadas

O repositório inclui docs/pytest_results_sprint2.txt, docs/smoke_test_results_sprint2.txt e docs/VALIDACAO_SPRINT2.md, com a distinção entre o que foi testado e o que ainda precisa de execução manual.

4.2. Limitação da verificação

O registro npm não respondeu a este ambiente (EAI_AGAIN), impedindo a instalação de React/Vite para o build. O Docker também não estava disponível. Portanto, não se afirma que as telas e a orquestração em contêineres tenham sido verificadas, apesar de o backend ter passado na integração por processos.

5. Como executar e demonstrar

Extraia o arquivo ZIP, abra a pasta microlearn no PowerShell e siga o roteiro completo presente no README. Um resumo dos comandos de execução é apresentado abaixo.

Ordem	Comando / atividade
1	Copy-Item .env.example .env
2	python -c "import secrets; print(secrets.token_urlsafe(48))"
3	Preencher JWT_SECRET, ADMIN_EMAIL e ADMIN_PASSWORD em .env
4	docker compose up --build -d
5	docker compose ps
6	Abrir http://localhost:5173 e http://localhost:8000/docs
7	Criar usuário, promover professor, publicar curso, matricular aluno

5.1. Execução dos testes

Para testar sem Docker, instale as dependências de backend/requirements.txt e o pytest, execute python -m pytest -q e depois python tests/smoke_processes.py. O segundo comando inicia e encerra cinco serviços HTTP temporários.

5.2. Próxima Sprint: Avaliações e Recomendações

O próximo incremento implementará questionários objetivos, registro das respostas, correção automática, consulta de notas e recomendação de aulas de revisão para notas inferiores a 7,0. Serão adicionadas as telas correspondentes e os testes de integração.

5.3. Referências do projeto

Documentos internos: proposta do MicroLearn, especificação de requisitos (Etapa 2), modelagem de arquitetura e componentes (Etapa 3), Product/Sprint Backlogs (Etapa 4), código da Sprint 1 e roteiros da disciplina de Engenharia de Software 2026.2. Este relatório descreve especificamente o código-fonte e os testes anexados ao ZIP da Sprint 2.